

Cron en Desarrollo

Tareas programadas — sintaxis, tipos y configuración en Next.js

Un **cron** es una tarea programada que se ejecuta automáticamente en momentos específicos o con una frecuencia determinada, sin que nadie la lance manualmente. El nombre viene de *cron* (del griego χρόνος — tiempo), el planificador de tareas de Unix.

¿Para qué se usa?

- Enviar emails programados (newsletters, recordatorios)
- Limpiar datos antiguos de la base de datos
- Generar informes periódicos
- Sincronizar datos con APIs externas
- Renovar cachés y procesar pagos recurrentes
- Hacer backups automáticos

La sintaxis cron

```
■ ■ ■ ■ minuto (0-59)
■ ■ ■ ■ hora (0-23)
■ ■ ■ ■ día del mes (1-31)
■ ■ ■ ■ mes (1-12)
■ ■ ■ ■ día de la semana (0-6, 0=domingo)
■ ■ ■ ■ ■
* * * * *
```

Ejemplos de expresiones

```
* * * * *      → cada minuto
0 * * * *      → cada hora (en punto)
0 9 * * *      → cada día a las 9:00
0 9 * * 1      → cada lunes a las 9:00
0 9 * * 1-5     → cada día laborable a las 9:00
0 0 1 * *      → el día 1 de cada mes a medianoche
0 0 * * 0      → cada domingo a medianoche
*/15 * * * *    → cada 15 minutos
0 9,18 * * *    → a las 9:00 y a las 18:00
```

Tipos de Cron

1. Cron del sistema operativo (Unix cron)

■ Cuándo usarlo: Servidores propios, VPS, cuando tienes acceso directo al sistema

El más básico. Se configura directamente en el servidor con *crontab*. No funciona en entornos serverless como Vercel o Netlify.

```
# Editar crontab
crontab -e

# Ejecutar script cada día a las 2:00
```

```
0 2 * * * /usr/bin/node /app/scripts/limpiar-bd.js

# Ejecutar cada hora
0 * * * * /usr/bin/python3 /app/scripts/sincronizar.py

# Ver crons activos
crontab -l
```

2. Cron con Vercel

■ **Cuándo usarlo:** Proyectos desplegados en Vercel — la opción más sencilla para Next.js

Vercel tiene soporte nativo para crons. Defines la frecuencia en *vercel.json* y apunta a una API Route. El plan gratuito permite crons cada hora como mínimo.

vercel.json

```
{
  "crons": [
    {
      "path": "/api/cron/limpiar-sesiones",
      "schedule": "0 2 * * *"
    },
    {
      "path": "/api/cron/enviar-newsletter",
      "schedule": "0 9 * * 1"
    }
  ]
}
```

API Route del cron

```
// app/api/cron/limpiar-sesiones/route.ts
export async function GET(request: Request) {
  // Verificar que viene de Vercel
  const authHeader = request.headers.get('authorization')
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return Response.json({ error: 'No autorizado' }, { status: 401 })
  }

  const eliminadas = await prisma.session.deleteMany({
    where: { expires: { lt: new Date() } }
  })

  return Response.json({ ok: true, eliminadas: eliminadas.count })
}
```

3. Cron con node-cron

■ **Cuándo usarlo:** Servidores Node.js propios, Express, APIs standalone

Librería que permite definir crons directamente en el código Node.js. Requiere que el proceso esté siempre activo — no funciona en serverless.

```
npm install node-cron
```

```
import cron from 'node-cron'

// Cada día a las 3:00
cron.schedule('0 3 * * *', async () => {
  await prisma.log.deleteMany({
    where: {
      createdAt: { lt: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000) }
    }
  })
})
```

```

    }
  })
})

// Cada 15 minutos
cron.schedule('*/*15 * * * *', async () => {
  await sincronizarStockConAPI()
})

// Solo días laborables a las 9:00
cron.schedule('0 9 * * * 1-5', async () => {
  await enviarResumenDiario()
})

```

4. Cron con Inngest

■ **Cuándo usarlo:** Next.js, Vercel, entornos serverless — el más potente para tareas complejas

Plataforma de background jobs y crons para entornos serverless. Muy popular con Next.js — soporta reintentos, monitorización y pasos encadenados.

```
npm install inngest
```

```

// inngest/functions.ts
export const enviarNewsletter = inngest.createFunction(
  { id: 'enviar-newsletter' },
  { cron: '0 9 * * 1' }, // cada lunes a las 9:00
  async ({ step }) => {
    const suscriptores = await step.run('obtener-suscriptores', async () => {
      return await prisma.usuario.findMany({ where: { newsletter: true } })
    })

    await step.run('enviar-emails', async () => {
      for (const usuario of suscriptores) {
        await resend.emails.send({ to: usuario.email, subject: 'Newsletter' })
      }
    })

    return { enviados: suscriptores.length }
  }
)

```

API Route para Inngest

```

// app/api/inngest/route.ts
import { serve } from 'inngest/next'
import { inngest } from '@inngest/client'
import { enviarNewsletter } from '@inngest/functions'

export const { GET, POST, PUT } = serve({
  client: inngest,
  functions: [enviarNewsletter],
})

```

5. Cron con QStash (Upstash)

■ **Cuándo usarlo:** Proyectos que ya usan Upstash Redis, necesitas crons + colas en el mismo sitio

Del mismo equipo que Upstash Redis. Colas de mensajes y crons para serverless.

```
npm install @upstash/qstash
```

```
import { Client } from '@upstash/qstash'

const client = new Client({ token: process.env.QSTASH_TOKEN! })

// Ejecutar en 1 hora
await client.publishJSON({
  url: 'https://tuapp.com/api/procesar-pedido',
  delay: 3600,
  body: { pedidoId: '123' }
})

// Cron recurrente
await client.schedules.create({
  destination: 'https://tuapp.com/api/cron/backup',
  cron: '0 2 * * *',
})
```

6. Cron con BullMQ + Redis

■ Cuándo usarlo: Apps grandes con servidor propio, necesitas reintentos y monitorización avanzada

El más potente para colas de tareas con crons integrados. Requiere servidor propio y Redis.

```
npm install bullmq ioredis

import { Queue, Worker } from 'bullmq'
import { Redis } from 'ioredis'

const connection = new Redis(process.env.REDIS_URL!)
const emailQueue = new Queue('emails', { connection })

// Añadir cron a la cola
await emailQueue.add(
  'newsletter-semanal',
  { tipo: 'newsletter' },
  { repeat: { pattern: '0 9 * * 1' } } // cada lunes a las 9:00
)

// Worker que procesa la cola
const worker = new Worker('emails', async (job) => {
  if (job.name === 'newsletter-semanal') {
    await enviarNewsletter()
  }
}, { connection })

worker.on('completed', (job) => console.log(`Job ${job.id} completado`))
worker.on('failed', (job, err) => console.error(`Job ${job.id} falló:`, err))
```

Tabla comparativa

Tipo	Entorno	Reintentos	Monitorización	Ideal para
Unix cron	Servidor propio	■	■	Scripts simples en VPS
Vercel crons	Vercel	■	■	Apps Next.js en Vercel
node-cron	Node.js	■	■	Servidores Express/Node
Inngest	Serverless	■	■	Next.js, tareas complejas
QStash	Serverless	■	■	Proyectos con Upstash
BullMQ	Servidor + Redis	■	■	Apps grandes con servidor

¿Cuál usar en Next.js?

Proyecto en Vercel sencillo	→ Vercel crons
Necesitas lógica compleja	→ Inngest
Ya usas Upstash Redis	→ QStash
Servidor Node.js propio	→ node-cron
App grande con servidor + Redis	→ BullMQ